SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

CO2 - ASSESSMENT TOOL - 1

SCENARIO-BASED ASSESSMENT

Course Code	**CSA3717**	**Course Title**	**Software Testing**
CO Assessed	**CO2**	**Assessment**	**Assessment Tool 1: Scenario Based Assessment**
Weightage	**50% of CIA**	**Total Marks**	**100**
CO2 Statement	*Design effective test cases using black-box, white-box testing techniques and to control methods flow.***(BL3)**		
Student Name	**B Reddy Balu**	**Reg. No.**	**192312499**
Section/Batch	**Slot A / 2023**	**Date**	**30-10-2026**

Code of Conduct:

I ***B Reddy Balu (192312499),*** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Hybrid Black Box and White Box Testing Strategy for a Financial Application

Scenario-Based Analysis: Why defects persist in production and how to prevent them

1. Problem Statement and Approach

A financial application is tested using both black box and white box techniques, yet defects keep escaping into production. In a financial system, such defects are expensive: a wrong rounding rule, an off-by-one limit check or an unhandled branch can mean incorrect balances, failed or duplicate transfers, regulatory exposure and loss of customer trust.

This report (i) critically analyses how the existing test case design strategies were integrated, (ii) evaluates whether test adequacy criteria and coverage metrics were applied properly, (iii) proposes an improved hybrid strategy combining Equivalence Class Partitioning (ECP), Boundary Value Analysis (BVA), Control Flow Graphs (CFG) and path testing, (iv) justifies how it improves coverage and defect prevention, and (v) concludes with a reasoned solution.

Note on scope: *the scenario does not provide project data (coverage reports, defect logs). The analysis therefore identifies the most probable root causes found in such projects, and the proposal is demonstrated on a representative module: a fund-transfer validation function. The same method applies to loan eligibility, interest calculation, payment authorisation and similar modules.*

1.1 Representative module used for illustration

The module validates a transfer request using four inputs (account status, amount, available balance, OTP result). Business rules:

- The source account must be ACTIVE (dormant, frozen or closed accounts are rejected).
- The amount must lie between 0.01 and 100,000 inclusive.
- The amount must not exceed the available balance.
- Amounts above 50,000 require a valid OTP; amounts of 50,000 or less do not.

2. Critical Analysis of the Existing Test Design Integration

Using both techniques is not the same as integrating them. Black box and white box testing find different classes of defects, so they only add value when each is designed to cover the other's blind spots. In projects where defects still escape, the usual pattern is that the two were run as parallel, uncoordinated activities. The main weaknesses are analysed below.

Weakness	What typically happens	Consequence in production
Siloed execution	Testers write black box cases from requirements; developers write unit tests for code. Neither team reviews the other's cases, so there is overlap in easy areas and gaps in hard ones.	Duplicated effort on simple flows; complex rule combinations remain untested.
Shallow black box design	ECP is applied only to valid and invalid inputs per field, with no boundary values (BVA) and no combination of classes across fields.	Off-by-one errors at limits (e.g. 100,000 vs 100,000.01, or exactly 50,000) pass testing.
White box limited to statement coverage	Unit tests only ensure that each line runs. Decisions with compound conditions or implicit else-paths are not forced both ways.	Untaken branches and short-circuited conditions hide logic faults.
No traceability	Requirements, black box cases and code coverage are not linked, so nobody can tell which rules are untested.	Missing or misunderstood requirements are never detected.
Specification gaps not checked	White box tests prove the code does what it does, not what it	Both 'missing logic' and 'extra logic' defects survive.

Weakness	What typically happens	Consequence in production
	should do. Black box tests do not reveal unreachable or dead code.	
Weak test data and environment	Tests use clean, small and unrealistic data; monetary precision, currencies, time zones, concurrency and legacy interfaces are ignored.	Rounding, race-condition and integration defects appear only on live data.
No feedback loop	Production defects are fixed but not analysed to improve test design or add regression cases.	The same defect classes recur.

Key observation: black box testing answers 'does the system do what the requirement says?', while white box testing answers 'has all of the code logic been exercised?'. A project that treats them as independent checklists leaves the intersection unverified: the code paths that implement a specific requirement under a specific boundary condition. That intersection is exactly where financial defects concentrate.

3. Evaluation of Test Adequacy Criteria and Coverage Metrics

A test adequacy criterion defines when testing is 'enough'. A coverage metric measures progress against that criterion. The typical project fails in the following ways.

3.1 Misuse of coverage metrics

- **Statement coverage treated as proof of quality.** Statement coverage is a weak criterion. A single test can execute every statement of an if-statement with no else-part while never testing the false outcome. Branch (decision) coverage subsumes statement coverage and should be the minimum for financial logic.

- **Aggregate percentages hide risky gaps.** A line coverage figure of 85–90% says nothing about the 10–15% left out. In financial modules this remainder is often the exception handling, rollback and limit-checking code, which is the highest-risk code.
- **Compound conditions not examined.** A condition such as (amount >= 0.01 AND amount <= 100000) has several ways to fail. Branch coverage needs only two tests, yet each atomic condition can hide a fault. Condition coverage, or MC/DC for critical decisions, is needed.
- **Path coverage and complexity ignored.** Statement and branch coverage do not ensure that different combinations of decisions along a route are tested. Cyclomatic complexity V(G) gives the number of linearly independent paths that must be tested, and this was not used to size the test effort.
- **Executed is not verified.** Coverage shows code was executed, not that results were checked. Tests without strong assertions inflate coverage yet detect nothing.

3.2 Missing black box adequacy criteria

- There was no stated criterion such as 'every equivalence class is covered at least once and every boundary is tested on and either side of the limit'.
- No requirement coverage matrix existed, so untested business rules were invisible.
- No fault-based adequacy measure (such as mutation score) was used to test the quality of the tests themselves.

3.3 Verdict

Test adequacy criteria and coverage metrics were not properly applied. They were used as reporting numbers rather than as design drivers and exit criteria. The project measured how much code was run, not how well the requirements and the decision logic were exercised. This directly explains why defects continue to appear in production despite 'both techniques being used'.

4. Proposed Improved Hybrid Testing Strategy

The proposed strategy is a single, traceable design flow in which black box analysis defines what must be tested, white box analysis verifies how thoroughly the code has been exercised, and the results of each are used to correct the other.

4.1 Strategy overview

1. **Step 1 – Requirement analysis and risk ranking.** Break every requirement into testable business rules and give each an ID (e.g. R1: account must be ACTIVE). Identify inputs, outputs and risk level (financial impact × likelihood).
2. **Step 2 – Black box design (what to test).** Apply ECP to every input, then BVA to every ordered range. Combine classes using a decision table for interacting rules.
3. **Step 3 – White box model (how the code is structured).** Draw the CFG for each critical unit, count decisions, compute V(G) = E − N + 2 (or decisions + 1) and list the basis paths.
4. **Step 4 – Merge and map.** Map each black box case to the basis path(s) it exercises. Any path with no case is a gap; add a test derived from path testing.
5. **Step 5 – Execute and measure.** Execute with coverage instrumentation. Compare against the adequacy criteria in Section 4.5. Investigate every uncovered branch: it is either a missing test, dead code or an unspecified behaviour.
6. **Step 6 – Validate the tests and close the loop.** Use mutation testing on the critical modules to confirm the tests actually detect faults. Feed every production defect back into the suite.

4.2 Black box design: ECP and BVA applied to the module

Equivalence classes (E = valid, I = invalid):

Input	**Valid classes**	**Invalid classes**
Account status	E1: ACTIVE	I1: DORMANT; I2: FROZEN; I3: CLOSED
Amount	E2: 0.01 to 50,000 (no OTP); E3: 50,000.01 to 100,000 (OTP)	I4: amount ≤ 0; I5: amount > 100,000; I6: non-numeric or more than 2 decimals

Input	Valid classes	Invalid classes
Balance vs amount	E4: amount ≤ balance	I7: amount > balance

Boundary values (using the on-boundary and both-sides approach, since defects in financial limits are mostly off-by-one):

Boundary	Test values	Expected outcome
Minimum amount (0.01)	0.00 | 0.01 | 0.02	Reject | Accept | Accept
Maximum amount (100,000)	99,999.99 | 100,000.00 | 100,000.01	Accept (OTP) | Accept (OTP) | Reject
OTP threshold (50,000)	49,999.99 | 50,000.00 | 50,000.01	No OTP | No OTP | OTP required
Balance limit	balance − 0.01 | balance | balance + 0.01 (as amount)	Accept | Accept | Reject

4.3 White box design: CFG, cyclomatic complexity and path testing

The validation logic contains five decision nodes (D1 to D5). Its CFG is shown in Figure 1.

Figure 1: Control Flow Graph of the fund-transfer validation module (V(G) = 6)

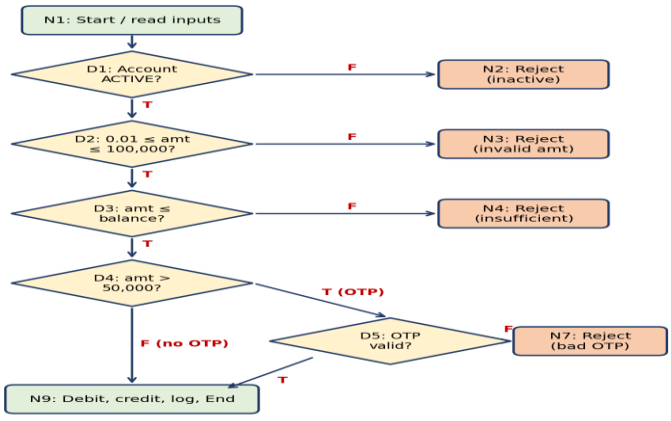


Figure 1: Control Flow Graph of the fund-transfer validation module

Cyclomatic complexity: V(G) = number of decisions + 1 = 5 + 1 = 6. Therefore six linearly independent (basis) paths must be tested. Using the formula E − N + 2 on the graph gives the same result, confirming the graph is consistent.

Path	Route through the CFG	Outcome	Test derived (illustrative)
P1	N1 → D1(F) → N2	Reject: inactive	Status = FROZEN, amount 500
P2	N1 → D1(T) → D2(F) → N3	Reject: invalid amount	ACTIVE, amount 0.00
P3	N1 → D1(T) → D2(T) → D3(F) → N4	Reject: insufficient	ACTIVE, amount 800, balance 799.99
P4	N1 → D1(T) → D2(T) → D3(T) → D4(F) → N9	Success, no OTP	ACTIVE, amount 50,000, balance 60,000
P5	… D4(T) → D5(T) → N9	Success with OTP	ACTIVE, amount 50,000.01, valid OTP
P6	… D4(T) → D5(F) → N7	Reject: bad OTP	ACTIVE, amount 75,000, wrong OTP

For decision D2, which is compound (amount ≥ 0.01 AND amount ≤ 100,000), condition coverage is added: one test must make the first atomic condition false, and another must make the second false, while MC/DC requires showing that each condition independently affects the decision outcome. The BVA values 0.00 and 100,000.01 from Section 4.2 satisfy this directly, which shows how the two techniques reinforce each other.

4.4 Merged test suite (extract)

The table links each test to the rule it verifies (traceability), the black box technique that produced it, and the path it covers. Tests that map to no path or no rule are reviewed.

ID	Rule	Input (status, amount, balance, OTP)	Technique	Path	Expected
T1	R1	FROZEN, 500, 1,000, –	ECP (I2)	P1	Reject
T2	R2	ACTIVE, 0.00, 1,000, –	BVA	P2	Reject
T3	R2	ACTIVE, 0.01, 1,000, –	BVA	P4	Accept
T4	R2	ACTIVE, 100,000.01, 200,000, valid	BVA	P2	Reject
T5	R3	ACTIVE, 800, 799.99, –	BVA	P3	Reject
T6	R3	ACTIVE, 800, 800.00, –	BVA	P4	Accept
T7	R4	ACTIVE, 50,000.00, 60,000, –	BVA	P4	Accept, no OTP
T8	R4	ACTIVE, 50,000.01, 60,000, valid	BVA	P5	Accept
T9	R4	ACTIVE, 75,000, 90,000, wrong	ECP (I8)	P6	Reject
T10	R4	ACTIVE, 75,000, 90,000, expired	ECP (I9)	P6	Reject
T11	R2	ACTIVE, 100.005, 1,000, –	ECP (I6)	P2	Reject (precision)

4.5 Adequacy criteria and exit conditions

Criterion	Target	Applies to
ECP: every valid and invalid class covered	100%	All modules
BVA: on-point and both neighbours of every limit	100%	All numeric and date limits

Criterion	Target	Applies to
Requirement coverage (traceability matrix)	100% of rules have at least one test	All modules
Statement coverage	100% (minimum floor)	Critical modules; ≥ 90% others
Branch / decision coverage	100%	Critical financial modules
Basis path coverage (V(G) paths)	100%	Modules with V(G) ≥ 4, and all payment/interest logic
Condition coverage or MC/DC	100% for decisions with compound conditions	Authorisation, limit and eligibility rules
Mutation score	≥ 80%	Critical modules (confirms test strength)
Open critical or high defects	Zero	Release gate

These targets are used as release gates in the CI/CD pipeline: a build that falls below any threshold for a critical module is not promoted.

4.6 Supporting practices

- Risk-based prioritisation: the highest-risk modules (payments, interest, reconciliation) get the deepest techniques; low-risk screens get lighter coverage.
- Test data management: use masked production-like data, cover decimal precision and rounding, multiple currencies, time zones and month-end/leap-year dates.
- Integration and non-functional testing: add interface contract tests, concurrency tests (double-submission, simultaneous debit) and performance tests, since unit-level hybrid tests cannot reveal them.
- Reviews: joint tester–developer review of the traceability matrix and CFGs before test execution.

- Regression automation: every test in Section 4.4 is automated and re-run on each change; each production defect adds a new regression test.
- Root cause analysis and defect escape metrics: measure the percentage of defects found in production and categorise by missed technique (missed boundary, missed path, requirement gap).

5. Justification: Better Coverage and Defect Prevention

Aspect	Existing approach	Proposed hybrid approach
Input-space coverage	Only valid and invalid values per field	ECP + BVA cover classes and the limits where financial defects cluster
Logic coverage	Statement coverage, branches skipped	Branch, condition/MC-DC and basis path coverage derived from the CFG
Test sizing	Arbitrary number of tests	V(G) gives a justified minimum number of paths
Blind spots	Each technique's gaps remain	Black box exposes missing logic; white box exposes untested and dead code
Traceability	None	Rule ↔ test ↔ path matrix; gaps visible
Test quality	Assumed from coverage	Verified with mutation testing and assertion review
Learning	Defects fixed only	Production defects drive new tests and process change

The approach improves coverage because the two techniques are made complementary. ECP and BVA are derived from the specification, so they detect missing or wrong behaviour even when the code has no corresponding branch. CFG-based path testing is derived from the implementation, so it detects logic that the specification never anticipated and shows which tests are redundant. Mapping one to the other creates a closed check: a rule with no path means missing code, a path with no rule means unspecified behaviour, and a path with no test means a testing gap.

It improves defect prevention because (a) defects are found earlier, at unit and integration level, where they are cheapest to fix; (b) the adequacy criteria act as objective release gates instead of opinion; (c) mutation testing prevents false confidence from high coverage; and (d) the feedback loop ensures that each escaped defect strengthens the suite, so the same defect cannot escape twice. Finally, risk-based depth ensures effort is spent where a failure would cost the most.

Limitations: path coverage grows rapidly with loops, so basis paths and risk-based selection are used instead of exhaustive paths; MC/DC and mutation testing add cost and are restricted to critical modules; and no test strategy removes the need for exploratory, security and performance testing. These trade-offs are accepted deliberately and documented.

6. Conclusion

Defects continue to reach production in the financial application not because black box and white box testing are ineffective, but because they were combined in name only. They were run independently, without shared traceability, with weak coverage criteria (mostly statement coverage), no boundary analysis and no check of test strength. Coverage figures were reported rather than used to drive test design and release decisions.

The recommended solution is a unified hybrid strategy: derive specification-based tests with ECP and BVA; model the code with CFGs and use cyclomatic complexity to define basis paths; map and reconcile both sets of tests through a traceability matrix; enforce explicit adequacy criteria (100% class and boundary coverage, branch and basis-path coverage, MC/DC for critical conditions, mutation score thresholds) as release gates; and feed every production defect back into the suite. Applied consistently, this gives measurable, defensible coverage of both what the system must do and how it does it, and it substantially reduces the likelihood of defects escaping into production.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario	
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts	
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions	
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach	
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification	